Akshay Adik

01-introduction

Final Dissertation Submission

Document Details

Submission ID
trn:oid:::3117:628506909

Submission Date
Sep 23, 2026, 6:35 PM GMT+5:30

Download Date
Sep 23, 2026, 6:38 PM GMT+5:30

File Name
01-introduction.pdf

File Size
173.2 KB

25 Pages

6,208 Words

34,550 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

1 Introduction

1.1 Background and Context

Micro-service-based distributed system has more than one services which interact with each other to achieve certain business goals. Such system can fail in production due to service fault or defect or runtime performance degradation due to heavy load. These faults can propagate across multiple dependent services. Resolving these defect in running system is often complex due to manual intervention, static diagnostic approaches and dynamic nature of containerized and cloud environment (Fu et al., 2025). Continuous integration and rapid deployment practices makes the environment more dynamic and defect patterns can shift within hours, producing fault signatures that differ from those observed in earlier releases. Identifying the root cause in these dynamic environment has become a moving target.

Software architecture has transitioned from monolithic applications to micro-services architecture. A monolith is a single, unified deployable unit, whereas a micro-service is a collection of small, independent services. Software engineers face scalability and deployment challenges in monolithic applications. Micro-services offer flexibility and modular services that perform specific functions. This architecture has helped reduce the dependency on single services. In addition, it is scalable when coupled with containerized or cloud systems. With multiple services, problems in a part of the service will only impact that functionality without affecting the rest of the application (Nobre et al., 2023). Advanced deployment tools, such as Docker and Kubernetes, provide flexibility in the deployment of micro-services. Docker is a lightweight runtime container environment, whereas Kubernetes is a container orchestration tool. This has helped major enterprises such as Amazon, Netflix, and Uber to have a decompose architecture with elasticity (Jonnaikode, 2025).

Architectural separation has helped teams choose their technology stack, which creates heterogeneity across products. However, this also leads to a lack of standardization across teams and incurs operational challenges (Nobre et al., 2023).

Service complexities have increased owing to the autonomous nature and flexibility of choosing technologies. Services use multiple API's and schema structures, resulting in service interconnections and frequent deployments. Every service attempts to implement reliability, which may result in fault-tolerant implementation. This may obscure the issue until it results in significant failure (Z. Chen et al., 2020). Defects in micro-services environment is often different from monolith environment. Micro-services has chained service interaction which is often dynamic. Failure in one service may cascade across API chain or message queue. Origin of the defect may be located several hopes away from point of observable degradation (Noor, 2021). Workload pattern can uncover hidden defects in services. Services that function properly under normal workloads may have several hidden issues under heavy workloads. The monitoring of these services is based on a fixed threshold. Sometimes, it misses the dynamic behavior of the application. Machine Learning models can help predict unusual patterns in applications, but models trained under certain workload data may skip actual issues caused by altogether different application usage patterns (Shivakumar, 2025).

Monitoring should cover all touch points in a multi-cloud environment with a service mesh infrastructure. Meeting the service-level agreement in such a heterogeneous environment requires proactive anomaly detection capability. Generative AI models can assist here. These models work on multi-modal data, such as logs, traces, and metrics, and quantify the distributed system state as a unified context for root cause analysis of defects (T. Wang & Qi, 2024). A huge volume of multi-

modal data is generated in cloud observability. Establishing a causal link with the input request helps identify the actionable insights required for root cause analysis (Gan et al., 2023).

Micro-services provide modular and adaptable architectures. It speeds up the deployment process without affecting the majority of applications. It also offers scalability and helps boost system performance. However, multi-service interactions create a complexity layer. Identifying the root cause of complex service interactions is difficult. The data required to identify the root cause are extensive and cover logs, traces, and metrics. Statistical tools or artificial intelligence can help to harvest data and extract unusual patterns. These patterns can help build cause-and-effect relationships with defects and system environments.

1.2 Problem Motivation

1.2.1 Nature of Failures in Micro-service Environment

Due to inherent complexity, different failures can be observed in micro-services system. This section will discuss different types of failures.

Cascading Failures. Services can be interdependent on each other. For example Service A can be dependent on B and B in turn depend on C. User may be interacting with service A but success of service A depends on success of B and C as well. This propagative characteristics can propagate the local faults through the entire system. Failure can be observed in service A but root cause can be at B or C. The boundary between the root cause and symptoms becomes blurred. Anomalies can quickly propagate across service invocation chain (J. Chen et al., 2024; Jonnaikode, 2025; Sarda et al., 2023; T. Wang & Qi, 2024; S. Xie et al., 2026). This has a cascading effect

on latency, service unavailability, and Quality Of Service (QoS) (Lee et al., 2023; Liu et al., 2021; Sarda et al., 2023; Shetty et al., 2024; Somashekar et al., 2024; S. Zhang et al., 2023).

It creates a back-pressure effect,"where poor performance can spread across the interaction chain (Gan et al., 2022a; Gan et al., 2023). These faults are destructive, create dependency complexity, and propagate quickly (T. Wang et al., 2024; T. Wang & Qi, 2024). Failure in one micro-service delays or fails all other micro-services in the invocation chain (Lee et al., 2023). For example, Failure in Shipping service can propagate to other services and in turn affect front-end application service. Anomalies can also have an indirect impact, where resource congestion caused by one service located on a certain host can affect all independent services on that host machine (Ma et al., 2020; Wang et al., 2021). Multiple dependent bottlenecks due to the same failure problem can create failures. E.g. Resource contention on shared VM (Somashekar et al., 2024). The propagative pattern is the primary driver of RCA complexity. Engineers must trace back convoluted or misleading failure paths and correlate the data to identify the origin of the failure (T. Wang & Qi, 2024; W. Zhang et al., 2024).

Diversity and Granularity. Failures can be due to kernel faults, software bugs, network failures, resource constraints, over-utilization, performance issues, configuration issues, or non-traditional failures related to a distributed environment (Sarda et al., 2023). These failures cover multiple application layers and are highly diverse in nature. Anomalies are deviations from the expected behavior and are often caused by different types of failures. These failures are systematically categorized at the granular level in the micro-service environment. For example, application, service, instance, pod, container, and node (Fu et al., 2025; Han et al., 2024; X. Zhang et al., 2025).

These failures are shown in the monitoring data as performance anomalies, such as saturation, I/O bottleneck, high CPU utilization, or explicit status anomalies, such as HTTP 4XX or 5XX errors (Aktaş & Kilinc, 2024; Ji et al., 2025; Tang et al., 2025). Common failure type includes CPU exhaustion, memory leak/hog, network delay/jam/loss/partitions, Java Virtual Machine (JVM) related problems or API logic faults (Ji et al., 2025; Pei et al., 2025; Pham et al., 2024; W. Zhang et al., 2024; Zhu et al., 2024). Micro-service-specific failures include configuration faults, such as JVM or Docker resource limits, multi-instant faults due to a lack of coordination or inconsistent state of the same service, and asynchronous execution faults due to the incorrect order of micro-service invocations (Zhou et al., 2019). Additionally, external activities, such as database migration, can also cause failures (S. Zhang et al., 2026). RCA should be capable to handle heterogeneous failure, identify the origin and explain the failure at granular level (Tao et al., 2024; S. Xie et al., 2026; S. Zhang et al., 2026).

Dynamic Systems and Fragmented Data. Micro-services systems are dynamic. Invocation paths or service interaction can change quickly. For example, Service A dependent on Service B can change the dependencies to Service C. Each service has its own monitoring, logging and tracking system in place. Cluster of multiple services generate huge logs, traces and metric data. Cloud infrastructure has made deployment and scalability easier than before. Effective RCA in such environment is influenced by these architectural challenges which can lead to delays in diagnosis, alerts overburden, and low granular results. Due to continuous delivery practices, such as frequent updates, the environment is dynamic and volatile (Aktaş & Kilinc, 2024; J. Chen et al., 2024; Heinrich et al., 2017; Ma et al., 2020; L. Wang et al., 2023; Yu et al., 2021; Zhou et al., 2019). Identifying normal behavior in such environments is difficult and can create false alarms (Z. Chen

et al., 2020; Heinrich et al., 2017; Nobre et al., 2023; Wang et al., 2021; Xing et al., 2025). Architecture of multi-tier environment is decomposed into multiple layers such as presentation, API, service, messaging and data tier. Fault originated in one tier can propagate via chain and create a flood of homogeneous incidents. Observability of the services becomes fragmented when services are collecting logs, traces and metrics in silos. Relying on single-modal data can create blind spots where some failure patterns can be skipped (Huang et al., 2024; Tao et al., 2024; S. Xie et al., 2026; Yu et al., 2023; S. Zhang et al., 2023). For example, failure may not be shown in the metric data but can be seen in the log data (Zhao et al., 2023). Pain points observed in RCA includes,

Dynamic Dependency. The dependency graph changes over time (L. Wang et al., 2023).

Telemetry Noise. Monitoring creates a huge volume of data, and sometimes real failures are buried in this data (Yu et al., 2023).

Coarse-grained result. RCA localization requires a Service Reliability Engineer (SRE) to spend additional time troubleshooting the service to identify the function-level or metric-level root cause (Wang et al., 2024; Yu et al., 2023; Zhu et al., 2024).

Metric data integrity. SRE may collect data selectively, resulting in partial or incomplete diagnosis (Jiang et al., 2023).

To overcome these pain points, researchers have used AIOps and machine learning techniques with the use of multi-modal data (logs, traces, metric,s etc.) to identify anomalous patterns (Hu et al., 2025; Lee et al., 2023; S. Zhang et al., 2023). This has helped in fine-grained root cause

analysis with minimal downtime (Wang et al., 2024; S. Xie et al., 2026; Yu et al., 2023; Zhu et al., 2024).

1.3 Significance of Root Cause Analysis

1.3.1 Mean-Time-To-Resolution (MTTR) and Economic Loss

RCA can helps in reducing mean time to resolution which translate into substantial economic saving and maintaining high Service Level Objections (SLO). Production incidents disrupt the normal operation of the system, which can lead to unsatisfied end users and economic losses. RCA helps minimize the gap between system incidents and recovery. This helps minimize the MTTR. Research shows that IT downtime costs $26.5 billion annually in lost revenue (C. Chen et al., 2023). In addition, it costs reputational damage, and immediate incident resolution can help reduce damage. For example, the RCA system and TraceDiag integrated into the Microsoft M365 Exchange system helped reduce the RCA time by 96.5% (Ding et al., 2023). Another RCA system, KPIRoot+, deployed in a cloud environment, has shown an improvement in root cause localization from 30 min to 5 min (Gu et al., 2025). CloudRCA deployed in Alibaba helped engineers reduce the time spent on failure by more than 20% (Y. Zhang et al., 2021). This shows that RCA can help minimize system failure and reduce downtime. This directly contributes to reducing reputation damage and financial losses caused by failure (C. Chen et al., 2023).

1.3.2 Explainability and Developer Trust

RCA should provide interpretation and explainable results for site reliability engineers (SRE) or on-call engineers (OCE) (Y. Chen et al., 2023). The trust of engineers in the RCA system's results will enable them to take the necessary mitigation actions. Machine learning or deep learn-

ing models are important for automated RCA, but the outputs produced by these models should be explainable. Explainability is important for justifying the root cause of failure and identifying scenarios that lead to performance degradation (Harutyunyan et al., 2023). Engineers require clear and transparent reasoning to trust and use the results of automated RCA. The RCA results are not interpretable, and SRE are often skeptical of accepting the RCA output for fear of losing repair time or worsening the fault. The system leverages explainability in different ways. RCACopilot and SynergyRCA use large language models to generate human-readable explanations of root causes with clear action steps (Y. Chen et al., 2023; Shan et al., 2025; Xiang et al., 2025). Groot makes casual relationship understandable. It provide the visual representation that shows how different system component are causally connected with root cause (H. Wang et al., 2021). The SAGE system has gone beyond the diagnosis. It can help in operational decision making. It generates actionable decisions, such as identifying resources with high utilization (Gan et al., 2021, 2022b). Rule induction systems, such as Chain-of-Event provide systems behavior as related events. These system provide parameters that SRE can tune as per their domain knowledge. It provides the likelihood of one event causing another (Yao et al., 2024). This helps in informed judgment rather than binary decisions. Interpretability and explainability are necessary features of an RCA system. This builds trust in the AIOps system. It helps reduce manual diagnosis effort and provides crucial insights that may be left by manual interpretation, and creates knowledge for further system enhancement to avoid future failure (Harutyunyan et al., 2023; Soldani & Brogi, 2023).

1.3.3 Industrial Adoption

The success of RCA systems can be measured in terms of their operational efficiency and adoption in production environments, such as cloud services or Kubernetes clusters. RCA system

must scale effectively with size and complexity of micro-service environment. It should process large volume of logs, metrics and traces (Fu et al., 2025; Gu et al., 2025; Pham et al., 2024). So, much of latest research is focused on demonstrating if approach can work in production environment where traditional or statistical methods fails to perform (T. Wang et al., 2024; T. Wang & Qi, 2024). This shift is reflected in several deployment like Microsoft's RCA copilot and eBay's LatentScope (Y. Chen et al., 2023; Z. Xie et al., 2024). TraceDiag has been integrated into the Microsoft M365 Exchange system (Ding et al., 2023). BALANCE is deployed in production to handle real-world RCA problems (C. Chen et al., 2023). SynergyRCA demonstrated its dependability in two real-world Kubernetes cluster scenarios. It has achieved an average diagnostic time of two minutes (Xiang et al., 2025). KylinRCA is designed for practical deployment in production and is packaged as a containerized application for rapid cloud deployment using Helm charts (Jian, n.d.). This indicates that the adoption of AI tools and technology in root cause analysis is not limited to theoretical studies but is also becoming part of the automated incident management process.

1.4 Problem Statement

The micro-service environment is dynamic, scalable, and heterogeneous. The organization has adopted the architecture and is using it in a production environment. This architecture increases the cost of incident resolutions owing to its complex structure and large volume of data. The root cause analysis of incidents is mainly manual or statistical rule-driven. Site reliability engineers must spend time on manual investigations. Micro-service environments can create cascading failures, their service dependency graphs can change quickly, and observability solutions are mostly siloed or service-specific. These conditions increase the mean time to detect (MTTD) and mean time to resolve (MTTR).

While recent developments in AI and machine learning have helped in building better solutions, they suffer from explainability, poor scalability, and weak multi-modal integration. Consequently, SRE or operational teams lack trust in these solutions. This study addresses the problems of AI-driven RCA approaches by studying Gen AI-based solutions. The main purpose of this study is to reduce the cost of defect management and improve operational efficiency by providing better explainability, scalability, and trust.

1.4.1 Breakdown of main problem

Data fragmentation and fusion. Observability data are specific to services or individual applications. Its volume and velocity are high. Combining and referring to this data for the entire application requires high cost and computation.

Causal Inference Problem. Dues to dynamic dependencies, identifying true root cause is challenging.

Explainability. AI-based RCA are black boxes by nature, and operations teams are often reluctant to trust their outputs.

When the volume of the data is high, the scaling RCA solution is often cost-sensitive and can create huge costs and processing power.

1.5 Research Aim

The primary aim of this research is to evaluate AI-driven approaches for root cause analysis in a production environment. These approaches are evaluated based on cost, applicability,

efficiency, and adoptability in a micro-service environment. The solution should be able to scale large-scale micro-services.

1.6 Research Objective

To define an advanced scalable approach for root cause analysis that will help reduce the mean time to detect and mean time to resolve the defect. This approach aims to integrate multi-modal data, such as traces, logs, and metrics generated by micro-services, and establish a causal link between the cause and effect. Secondary objectives are,

- To investigate how multi-modal data help in improving the accuracy of root cause analysis in micro-service environment
- Establish dynamic causal discovery and build the propagation path of fault diagnosis.
- Analyze the LLM models for root cause explanations and long-term fix.

1.7 Research Questions

- **RQ 1:** How can generative AI support root cause analysis, and what impact does it have on cost, mean time to resolve (MTTR) and effectiveness of fix?
- **RQ 2:** What are the operational challenges in adopting GenAI based RCA solutions?
- **RQ 3:** How can the effectiveness of the GenAI solution be assessed in comparison with the statistical and traditional RCA approaches?
- **RQ 4:** How does the explainability of GenAI-based solutions influence the interpretation of the root cause output in the RCA workflow?

1.8 Testable Hypotheses

- **H1:** If a hybrid AI RCA system integrating statistical, graph-based, and generative reasoning is used in place of single-modality AI or statistical baselines, then Mean-Time-To-Resolution (MTTR) will be significantly reduced.
- **H2:** If a hybrid multi-pipeline architecture (statistical, graph, and LLM reasoning operating as peers) is used in place of any single reasoning pipeline, then root-cause diagnostic accuracy (HR@3) will improve.
- **H3a:** If the RCA system produces explainable output (e.g., Chain-of-Evidence narratives) in place of opaque/black-box output, then operations-team trust in the system's diagnoses will increase.
- **H3b:** If operations-team trust in the RCA system's output increases, then intent to adopt the technology will increase.
- **H4a:** If a hybrid AI RCA approach is used in place of single-modality alternatives, then Cost per Resolution (CpR) will be lower (Confirmatory — B-H7).
- **H4b:** If estate size increases from 100 to 5,000 services in the preregistered scaling sub-study, then Plane-2 latency and CpR will remain within the design-committed SLO bounds (Separate preregistered experiment, not part of the main 16k-run ablation).

1.9 Conceptual Framework

Framework consist of below building blocks,

- **Multi-modal observability data:** Metrics, logs, and trace data, along with other data generated by micro-services, help identify the state of the application.

- **Causal Inference:** Transform data helps in identifying the actionable insight using AI based model
- **Generative AI:** Generative AI models can help improve the explainability of solutions with clear action steps.

Together, these help build a better root cause analysis system in micro-services.

1.10 Limitation

The main limitation of this study is the publicly available dataset used to test the GenAI solution and compare it with traditional approaches.

- The performance of AI models may degrade owing to concept drift.
- Organization culture, and management may influence adoption outcome.
- Cost of training model could be high at initial stages.

1.11 Study Scope

This study focuses on a production-grade micro-service environment. Deployment is preferably performed using a containerized environment, such as Kubernetes. Scope will be as below,

- GenAI assisted approach
- Operational metric such as MTTR, MTTD and Cost
- Defect management and incidence response processes

1.12 Assumptions

- Incident data is available
- Practitioner posses baseline operational knowledge

1.13 Definition of Key Terms

Table 1
Definition of Key Terms

Concepts	Definition
Root Cause Analysis (RCA)	Method of identifying the underlying origin of a fault or defect.
Mean-Time-To-Resolution (MTTR)	Average time taken to resolve production issues.
Mean-Time-To-Detect (MTTD)	Average time taken to identify a production issue.
Cascading Failure / Chain Failure	A typical pattern in microservices where failures in one service propagate through dependency chains, potentially obscuring the original source of the fault.
Metrics	Structured quantitative time-series data related to CPU usage, resource utilization, latency, and other system performance indicators.
Logs	Semi-structured or unstructured data generated by services, including events, runtime information, and error data.
Traces	Hierarchical data that tracks the flow of a user request across services.

Concepts	Definition
Anomaly Detection (AD)	The first stage of the RCA lifecycle, involving the identification of deviations from expected patterns or behavior.
Root Cause Localization (RCL)	The process of identifying the location or source of a defect or failure.
Causal Inference / Causal Link	A statistical approach used to identify cause-and-effect relationships.
Generative AI (GenAI) / Large Language Models (LLMs)	Advanced artificial intelligence models used to generate human-readable explanations, reasoning, and remediation recommendations.
Hallucination	A situation in which a model generates factually incorrect information or a reasoning path, potentially resulting in incorrect explanations or remediation steps.
Concept Drift	The degradation of model accuracy over time due to changes in system behavior or underlying data patterns.
Alert Fatigue	A condition caused by a high volume of alerts, particularly false positives, which can reduce the effectiveness of operators.

References

Aktaş, K., & Kilinc, H. H. (2024). Interaction Prediction and Anomaly Detection in a Microservices-based Telecommunication Platform. *Proceedings of the 2024 International Conference on Software and Systems Processes*, 56–65. https://doi.org/10.1145/3666015.3666017

Chen, C., Yu, H., Lei, Z., Li, J., Ren, S., Zhang, T., Hu, S., Wang, J., & Shi, W. (2023). BALANCE: Bayesian Linear Attribution for Root Cause Localization. *Proceedings of the ACM on Management of Data*, *1*(1), 1–26. https://doi.org/10.1145/3588949

Chen, J., Zhang, R., Chen, P., Ren, J., Wu, Z., Wang, Y., Li, X., & Xiong, L. (2024). MTG_CD: Multi-scale learnable transformation graph for fault classification and diagnosis in microservices. *Journal of Cloud Computing*, *13*(1), 103. https://doi.org/10.1186/s13677-024-00666-0

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2023). *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents* (arXiv:2305.15778). arXiv. https://doi.org/10.48550/arXiv.2305.15778

Chen, Z., Kang, Y., Li, L., Zhang, X., Zhang, H., Xu, H., Zhou, Y., Yang, L., Sun, J., Xu, Z., Dang, Y., Gao, F., Zhao, P., Qiao, B., Lin, Q., Zhang, D., & Lyu, M. R. (2020). Towards intelligent incident management: why we need it and how we make it. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1487–1497. https://doi.org/10.1145/3368089.3417055

Ding, R., Zhang, C., Wang, L., Xu, Y., Ma, M., Wu, X., Zhang, M., Chen, Q., Gao, X., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1762–1773. https://doi.org/10.1145/3611643.3613864

Fu, N., Cheng, G., Teng, Y., Dai, G., Yu, S., & Chen, Z. (2025). Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives. *ACM Computing Surveys*, *57*(12), 1–37. https://doi.org/10.1145/3736755

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2021). *Sage: Using Unsupervised Learning for Scalable Performance Debugging in Microservices* (arXiv:2101.00267). arXiv. https://doi.org/10.48550/arXiv.2101.00267

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022a). Enabling Practical Cloud Performance Debugging with Unsupervised Learning. *ACM SIGOPS Operating Systems Review*, *56*(1), 34–41. https://doi.org/10.1145/3544497.3544503

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022b). Practical and Scalable ML-Driven Cloud Performance Debugging With Sage. *IEEE Micro*, *42*(4), 27–36. https://doi.org/10.1109/MM.2022.3169445

Gan, Y., Liu, G., Zhang, X., Zhou, Q., Wu, J., & Jiang, J. (2023). Sleuth: A Trace-Based Root Cause Analysis System for Large-Scale Microservices with Graph Neural Networks. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 324–337. https://doi.org/10.1145/3623278.3624758

Gu, W., Zhong, R., Yu, G., Sun, X., Liu, J., Huo, Y., Chen, Z., Zhang, J., Gu, J., Yang, Y., & Lyu, M. R. (2025). *KPIRoot+: An Efficient Integrated Framework for Anomaly Detection and Root Cause Analysis in Large-Scale Cloud Systems* (arXiv:2506.04569). arXiv. https://doi.org/10.48550/arXiv.2506.04569

Han, Y., Du, Q., Huang, Y., Wu, J., Tian, F., & He, C. (2024). The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 931–943. https://doi.org/10.1145/3691620.3695475

Harutyunyan, A., Poghosyan, A., Harutyunyan, L., Aghajanyan, N., Bunarjyan, T., & Vinck, A. J. H. (2023). Challenges and Experiences in Designing Interpretable KPI-diagnostics for Cloud Applications. *JUCS - Journal of Universal Computer Science*, *29*(11), 1298–1318. https://doi.org/10.3897/jucs.112570

Heinrich, R., Van Hoorn, A., Knoche, H., Li, F., Lwakatare, L. E., Pahl, C., Schulte, S., & Wettinger, J. (2017). Performance Engineering for Microservices: Research Challenges and Directions. *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, 223–226. https://doi.org/10.1145/3053600.3053653

Hu, J., Long, L., Sui, H., Gu, Z., & Zheng, G. (2025). CFTL: System Log Parsing Method Driven from Clustering According to First Token and Length for Anomaly Detection. *Applied Sciences*, *15*(4), 1740. https://doi.org/10.3390/app15041740

Huang, J., Chen, P., Yu, G., Wang, Y., Huang, H., & He, Z. (2024). *FaaSRCA: Full Lifecycle Root Cause Analysis for Serverless Applications* (arXiv:2412.02239). arXiv. https://doi.org/10.48550/arXiv.2412.02239

Ji, X., Zhang, L., Zhang, W., Peng, F., Mao, Y., Liao, X., & Zhang, K. (2025). LEMAD: LLM-Empowered Multi-Agent System for Anomaly Detection in Power Grid Services. *Electronics*, *14*(15), 3008. https://doi.org/10.3390/electronics14153008

Jian, H. (n.d.). *Research on fault diagnosis and root cause analysis based on full stack observability*.

Jiang, X., Pan, Y., Ma, M., & Wang, P. (2023). Look Deep into the Microservice System Anomaly through Very Sparse Logs. *Proceedings of the ACM Web Conference 2023*, 2970–2978. https://doi.org/10.1145/3543507.3583338

Jonnaikode, S. (2025). Detecting Software Anomalies in Microservices using Spectrograms, Scalograms, and Convolutional Neural Networks [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235008860). https://www.proquest.com/dissertations-theses/detecting-software-anomalies-microservices-using/docview/3235008860/se-2?accountid=25283

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023). *Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data* (arXiv:2302.05092). arXiv. https://doi.org/10.48550/arXiv.2302.05092

Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J., & Wu, Z. (2021). MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems. *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 338–347. https://doi.org/10.1109/ICSE-SEIP52600.2021.00043

Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., & Wang, P. (2020). AutoMAP: Diagnose Your Microservice-based Web Applications Automatically. *Proceedings of The Web Conference 2020*, 246–258. https://doi.org/10.1145/3366423.3380111

Nobre, J., Pires, E. J. S., & Reis, A. (2023). Anomaly Detection in Microservice-Based Systems. *Applied Sciences*, *13*(13), 7891. https://doi.org/10.3390/app13137891

Noor, A. I. (2021). Real-Time QoS Monitoring and Anomaly Detection on Microservice-Based Applications in Cloud-Edge Infrastructure [Ph.D., University of Newcastle Upon Tyne (United Kingdom)]. In *PQDT - UK & Ireland*. ProQuest Dissertations & Theses Global (2796835651). https://www.proquest.com/dissertations-theses/real-time-qos-monitoring-anomaly-detection-on/docview/2796835651/se-2?accountid=25283

Pei, C., Wang, Z., Liu, F., Li, Z., Liu, Y., He, X., Kang, R., Zhang, T., Chen, J., Li, J., Xie, G., & Pei, D. (2025). *Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis* (arXiv:2502.08224). arXiv. https://doi.org/10.48550/arXiv.2502.08224

Pham, L., Ha, H., & Zhang, H. (2024). Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We? *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 706–715. https://doi.org/10.1145/3691620.3695065

Sarda, K., Namrud, Z., & Rouf, R. (2023). *ADARMA: Auto-Detection and Auto-Remediation of Microservice Anomalies by Leveraging Large Language Models*.

Shan, A., Kaur, J., Singh, R., Banka, T., Yavatkar, R., & Sridhar, T. (2025). *RCA Copilot: Transforming Network Data into Actionable Insights via Large Language Models* (arXiv:2507.03224). arXiv. https://doi.org/10.48550/arXiv.2507.03224

Shetty, M., Chen, Y., Somashekar, G., Ma, M., Simmhan, Y., Zhang, X., Mace, J., Vandevoorde, D., Las-Casas, P., Gupta, S. M., Nath, S., Bansal, C., & Rajmohan, S. (2024). *Building AI Agents for Autonomous Clouds: Challenges and Design Principles* (arXiv:2407.12165). arXiv. https://doi.org/10.48550/arXiv.2407.12165

Shivakumar, M. (2025). Aiops–driven Adaptive Anomaly Detection in Evolving Cloud Environments using Transfer Learning [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235005709). https://www.proquest.com/dissertations-theses/aiops-driven-adaptive-anomaly-detection-evolving/docview/3235005709/se-2?accountid=25283

Soldani, J., & Brogi, A. (2023). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, *55*(3), 1–39. https://doi.org/10.1145/3501297

Somashekar, G., Dutt, A., Adak, M., Lorido Botran, T., & Gandhi, A. (2024). GAMMA: Graph Neural Network-Based Multi-Bottleneck Localization for Microservices Applications. *Proceedings of the ACM Web Conference 2024*, 3085–3095. https://doi.org/10.1145/3589334.3645665

Tang, P., Tang, S., Pu, H., Miao, Z., & Wang, Z. (2025). *MicroRCA-Agent: Microservice Root Cause Analysis Method Based on Large Language Model Agents* (arXiv:2509.15635). arXiv. https://doi.org/10.48550/arXiv.2509.15635

Tao, L., Zhang, S., Jia, Z., Sun, J., Ma, M., Li, Z., Sun, Y., Yang, C., Zhang, Y., & Pei, D. (2024). Giving Every Modality a Voice in Microservice Failure Diagnosis via Multimodal Adaptive Optimization. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1107–1119. https://doi.org/10.1145/3691620.3695489

Wang, H., Wu, Z., Jiang, H., Huang, Y., Wang, J., Kopru, S., & Xie, T. (2021). *Groot: An Event-graph-based Approach for Root Cause Analysis in Industrial Settings* (arXiv:2108.00344). arXiv. https://doi.org/10.48550/arXiv.2108.00344

Wang, L., Zhang, C., Ding, R., Xu, Y., Chen, Q., Zou, W., Chen, Q., Zhang, M., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Root Cause Analysis for Microservice Systems via Hierarchical Reinforcement Learning from Human Feedback. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5116–5125. https://doi.org/10.1145/3580305.3599934

Wang, T., & Qi, G. (2024). *A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends* (arXiv:2408.00803). arXiv. https://doi.org/10.48550/arXiv.2408.00803

Wang, T., Qi, G., & Wu, T. (2024). *KGroot: Enhancing Root Cause Analysis through Knowledge Graphs and Graph Convolutional Neural Networks* (arXiv:2402.13264). arXiv. https://doi.org/10.48550/arXiv.2402.13264

Wang, Y., Li, G., Wang, Z., Kang, Y., Zhou, Y., Zhang, H., Gao, F., Sun, J., Yang, L., Lee, P., Xu, Z., Zhao, P., Qiao, B., Li, L., Zhang, X., & Lin, Q. (2021). *Fast Outage Analysis of Large-scale Production Clouds with Service Correlation Mining* (arXiv:2103.03649). arXiv. https://doi.org/10.48550/arXiv.2103.03649

Wang, Y., Zhu, Z., Fu, Q., Ma, Y., & He, P. (2024). MRCA: Metric-level Root Cause Analysis for Microservices via Multi-Modal Data. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1057–1068. https://doi.org/10.1145/3691620.3695485

Xiang, Y., Chen, C. P., Zeng, L., Yin, W., Liu, X., Li, H., & Xu, W. (2025). *Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM* (arXiv:2506.02490). arXiv. https://doi.org/10.48550/arXiv.2506.02490

Xie, S., Wang, J., He, H., Wang, Z., Zhao, Y., Zhang, N., & Li, B. (2026). TVDiag: A Task-oriented and View-invariant Failure Diagnosis Framework for Microservice-based Systems with Multimodal Data. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–39. https://doi.org/10.1145/3734868

Xie, Z., Zhang, S., Geng, Y., Zhang, Y., Ma, M., Nie, X., Yao, Z., Xu, L., Sun, Y., Li, W., & Pei, D. (2024). Microservice Root Cause Analysis With Limited Observability Through Intervention Recognition in the Latent Space. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 6049–6060. https://doi.org/10.1145/3637528.3671530

Xing, S., Wang, Y., & Liu, W. (2025). Multi-Dimensional Anomaly Detection and Fault Localization in Microservice Architectures: A Dual-Channel Deep Learning Approach with Causal Inference for Intelligent Sensing. *Sensors*, *25*(11), 3396. https://doi.org/10.3390/s25113396

Yao, Z., Pei, C., Chen, W., Wang, H., Su, L., Jiang, H., Xie, Z., Nie, X., & Pei, D. (2024). Chain-of-Event: Interpretable Root Cause Analysis for Microservices through Automatically Learning Weighted Event Causal Graph. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 50–61. https://doi.org/10.1145/3663529.3663827

Yu, G., Chen, P., Chen, H., Guan, Z., Huang, Z., Jing, L., Weng, T., Sun, X., & Li, X. (2021). MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. *Proceedings of the Web Conference 2021*, 3087–3098. https://doi.org/10.1145/3442381.3449905

Yu, G., Chen, P., Li, Y., Chen, H., Li, X., & Zheng, Z. (2023). Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 553–565. https://doi.org/10.1145/3611643.3616249

Zhang, S., Jin, P., Lin, Z., Sun, Y., Zhang, B., Xia, S., Li, Z., Zhong, Z., Ma, M., Jin, W., Zhang, D., Zhu, Z., & Pei, D. (2023). *Robust Failure Diagnosis of Microservice System through Multimodal Data* (arXiv:2302.10512). arXiv. https://doi.org/10.48550/arXiv.2302.10512

Zhang, S., Xia, S., Fan, W., Shi, B., Xiong, X., Zhong, Z., Ma, M., Sun, Y., & Pei, D. (2026). Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Transactions on Software Engineering and Methodology*, *35*(1), 1–55. https://doi.org/10.1145/3715005

Zhang, W., Guo, H., Yang, J., Tian, Z., Zhang, Y., Yan, C., Li, Z., Li, T., Shi, X., Zheng, L., & Zhang, B. (2024). *mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture* (arXiv:2404.12135). arXiv. https://doi.org/10.48550/arXiv.2404.12135

Zhang, X., Wang, Q., Li, M., Yuan, Y., Xiao, M., Zhuang, F., & Yu, D. (2025). *TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems* (arXiv:2504.20462). arXiv. https://doi.org/10.48550/arXiv.2504.20462

Zhang, Y., Guan, Z., Qian, H., Xu, L., Liu, H., Wen, Q., Sun, L., Jiang, J., Fan, L., & Ke, M. (2021). CloudRCA: A Root Cause Analysis Framework for Cloud Computing Platforms. *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 4373–4382. https://doi.org/10.1145/3459637.3481903

Zhao, C., Ma, M., Zhong, Z., Zhang, S., Tan, Z., Xiong, X., Yu, L., Feng, J., Sun, Y., Zhang, Y., Pei, D., Lin, Q., & Zhang, D. (2023). Robust Multimodal Failure Detection for Microservice Systems. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5639–5649. https://doi.org/10.1145/3580305.3599902

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Liu, D., Xiang, Q., & He, C. (2019). Latent error prediction and fault localization for microservice applications by learning from system trace logs. *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 683–694. https://doi.org/10.1145/3338906.3338961

Zhu, Y., Wang, J., Li, B., Tang, X., Li, H., Zhang, N., & Zhao, Y. (2024). *Root Cause Localization for Microservice Systems in Cloud-edge Collaborative Environments* (arXiv:2406.13604). arXiv. https://doi.org/10.48550/arXiv.2406.13604